

Methodological Comparison with the ERA System of Aygün et al.

This document provides a point-by-point comparison between this project's Kaggle hybrid agent and the **ERA (Empirical Research Assistant, Google DeepMind)** system of **Eser Aygün et al. (2026, Nature; preprint arXiv:2509.06503v3, 78 pages)**: methods, benchmarks, results, and rigor — and it **honestly marks out what this project is "not," as well as where it is actually more rigorous.**

Citation conventions: on the ERA side, [ERA p.N] denotes the preprint page number; on this project's side, file:line denotes a location within the repo. Every number is traceable.

Contents

1. One-Sentence Positioning	1
2. Methodological Comparison	2
2.1 Search Engine	2
2.2 External-Idea Injection (Second Pillar) — the Most Critical Comparison	2
3. Benchmark Comparison	3
4. Results Comparison	4
5. Ablation and Statistical-Rigor Comparison	5
6. Honest Gap and Positioning Summary	6
Appendix: Paths to Truly Align with ERA's Second Pillar Once Wired	6

1. One-Sentence Positioning

- **ERA** is an **automated system**: given a machine-scorable task, the **LLM writes Python code directly**, which is executed and scored in a sandbox, and then a **tree search** climbs within the "code space"; external research ideas guide the LLM toward specific approaches via **prompt injection**. [ERA p.2, p.13]
- **This project** is a **Claude Code Skill + human-in-the-loop** competition workflow, augmented by a **self-built tree search**; the search performs a hand-written combinatorial search over the "**config space of a fixed GBDT family (LGB/XGB/CAT)**," with the **LLM not inside the mutation loop**. docs/TECH_REPORT.md:28-34 , docs/PREFACE.md:94-101

Both share the same "two-pillar" language (**LLM + tree search**; the second pillar = integration of external ideas), and this project's Stage 4/5 is designed precisely around these two pillars to perform a **controlled ablation**. But as described below, the two differ fundamentally in "what role the LLM plays," and the comparison must make this explicit and must not imply equivalence.

2. Methodological Comparison

2.1 Search Engine

Aspect	ERA	This project
Search type	PUCT tree search (the authors describe it as closer to Flat-UCB than AlphaZero; no rollout, expansion sampled from all tree nodes) [ERA p.13]	Candidate tree + backtracking combinatorial search (harness_v2/v3/v4); breadth-then-depth budget phase machine + explore burst + auto-stop docs/PREFACE.md: 131-137
A node is	a piece of LLM-generated code that is executed and scored [ERA p.13]	a config : solo (single model + single hyperparameter set, full K-fold) or blend docs/PREFACE.md: 94-96
Mutation operator	the LLM generates a new child program from the parent node's code + score ; randomness comes from LLM sampling [ERA p.13-14]	propose_child / select_next_parent traverse the SOL0_SEEDS seed set + mutation queue; boundary-push extrapolates the tuning-range boundary by one notch docs/phase_j_j3_findings.md: 37-39
Ensemble	decided by the LLM within its own code	OOF weight search (blend = weighted average of the solos' OOF, Dirichlet weights), mega-blend (ensemble of all solos) docs/PREFACE.md: 86, 101
LLM used	Gemini 2.5 Flash for all experiments (switching to Pro yields limited gains) [ERA p.13]	Claude Code (but only outside the loop for EDA/features/decisions and hand-written seeds , not generating code inside the search loop)
Tuning constant	c_puct = 1, tuned on the Kaggle benchmark [ERA p.13]	no corresponding hyperparameter; budget controlled by nodes/phases

Core difference (must be stated explicitly): ERA's tree search climbs over **arbitrary code generated by the LLM** — the mutation operator is literally "the LLM rewrites the program." This project's tree search performs a combinatorial/weight search over a **model family and tuning space that the human predefines** — the mutation operator is a hand-written propose_child. Therefore this project's Stage 4 aligns with ERA's **structural idea** of "**replacing a linear single path with a tree search**," **not** with the capability of "the LLM writing programs automatically." This must not be blurred in reports or external explanations.

2.2 External-Idea Injection (Second Pillar) — the Most Critical Comparison

Aspect	ERA	This project
Injection point	prepended to the prompt of "LLM-generated code" ; the LLM then writes code following that idea [ERA p.14]	merged into the tree['priors'] returned by suggest_priors (harness_v4's suggest_priors_v4) docs/phase_j_j3_findings.md: 15-22

Aspect	ERA	This project
Idea source	expert suggestions, LLM summaries of highly-cited papers/textbooks , Deep Research, AI co-scientist [ERA p.3,5,14]	external idea bank knowledge/idea_bank.md ([EXT]); internal experience library knowledge/experience.md ([INT]) README.md:48-51
Actually consumed?	Yes — the injection point is the input to the mutation operator, directly shaping each generated program [ERA p.12,14]	No — <code>tree['priors']</code> is write-only , used only for printing/logging; <code>SOLO_SEEDS /mutation queue/ propose_child / select_next_parent /evaluator none</code> of them read it docs/phase_j_j3_findings.md:5-42
Empirical evidence	literature-guided ERA produced BBKNN (TS), +14% over the best published method; 44% (24/55) of recombined solutions beat both parents [ERA p.5]	single-variable A/B (s3e3): <code>off ([INT])</code> vs <code>ext ([INT]+2 net-new [EXT])</code> → global best is node #11 in both, AUC exactly identical at 0.841442 , 22 nodes bit-for-bit identical ($\Delta=+0.00000000$) docs/phase_j_j3_findings.md:5-42
Idea recombination	the LLM verbalizes the principles of two parent methods and generates a new prompt (conceptual-space recombination) [ERA p.15]	<code>harness_v4</code> has a recombination mutation (synthesizing the core concepts of two high-scoring nodes), but because the consumption end above is not wired, it is likewise ineffective for the search

Conclusion: ERA's second pillar **carries load** — the injected text is directly the input to the mutation operator (the LLM prompt). This project's second pillar is a **"constructive no-op" under the current architecture**: the injection mechanism (pool merging/deduplication/gating) is itself correct, but the **consumption end is not wired**. The experience library's knowledge **did** guide the search, but **via the author's hand-written seeds** (the author reads experience.md and hand-writes priors into the seed configs; `[PRIOR PK]` is a hand-written string constant, and `prior_usage_summary` only regex-parses it without tracing back to `tree['priors']`). This is the largest methodological gap between this project and ERA, honestly recorded in docs/phase_j_j3_findings.md and docs/TECH_REPORT.md:90-96 . **Root cause:** the search loop of this project has no "LLM generates code" operator, so naturally there is no prompt into which to inject; to make injection carry load, one must first wire `tree['priors']` into `propose_child /seed` logic, or introduce LLM-in-loop code mutation.

3. Benchmark Comparison

Aspect	ERA	This project
Kaggle rounds	16 Playground events from the 2023 season (S3) (regression + classification) [ERA p.4, Table S2 p. 57]	15 events: 10 from the same season S3 + 5 cross-season S4-S6 docs/benchmark_facts.json:4-7
Scoring method	direct submission to the real Kaggle leaderboard , metric =	almost entirely local OOF (5-fold) ; only s3e16 has a real LB , the other 14 events are closed (403) with CV only docs/

Aspect	ERA	This project
	human percentile on the public leaderboard [ERA p.4]	TECH_REPORT.md: 52-55 , docs/SUMMARY_REPORT.md: 68-71
Comparison baselines	Single (1 call), Best-of-1000, AIDE, TS, TS+Expert, TS+BDT [ERA p.3]	this project's own ablation ladder: Stage 1 (no-skill baseline) → 2 → 3 → 4 → 5 docs/PREFACE.md: 104-143
Beyond Kaggle	scRNA-seq, COVID prediction, GIFT-Eval, geospatial, ZAPBench, numerical integration — 6 scientific domains [ERA p.5-9]	none (tabular competitions only)

Strong alignment point: this project's 10 S3 events (s3e1/3/5/7/9/11/14/16/19/20) **all fall within ERA's list of 16**; the 6 events this project cannot obtain (s3e2/4/8/10/12/17 — the account cannot accept the rules of these closed competitions, download returns 403) **are exactly those ERA has but this project lacks**. Therefore this project's Kaggle benchmark can be viewed as a **10/16 subset of the exact same benchmark as ERA**, only with scoring changed to local CV.

Key gap: ERA measures "reaching expert level" using the **human percentile on the real leaderboard**; apart from s3e16, this project has **no** real leaderboard and **no** human-percentile calibration. Therefore what this project can honestly claim is "**each added capability yields a step-wise improvement in the controlled CV score**," and it **cannot** claim "reached the human Nth percentile."

4. Results Comparison

	ERA	This project
Main Kaggle result	16-event average public-leaderboard percentile: Single $\approx$ 34% → Best-of-1000 $\approx$ 49% → AIDE $\approx$ 56% → TS$\approx$64% → TS+Expert$\approx$68% → TS+BDT$\approx$71% (read from Fig.1b, no numeric table) [ERA p.3-4]	15-event Stage 1→4 ladder holds in every event (9 up, 1 tie, no overall regression at any stage) docs/TECH_REPORT.md: 59-60
Improvement pattern	TS clearly beats single-call and best-of-1000, and also beats AIDE [ERA p.4]	improvements concentrated in structural-insight events (s3e20 +25.7% , s3e5 +19.2%), near-ceiling events <1% docs/SUMMARY_REPORT.md: 131
Real-leaderboard evidence	real leaderboard across the whole benchmark; scRNA 40 methods beat the human leaderboard leader, COVID 14 models beat the CDC ensemble [ERA p.1,5,7]	only s3e16: the tree-search version beats the linear champion in the same direction on both leaderboards (Public 1.34356→1.34315, Private 1.34075→1.33859) docs/SUMMARY_REPORT.md: 68-85
Generalization	reaches expert level across all 6 scientific domains [ERA p.5-9]	cross-season 5 events (5 metrics) — ladder still holds, improvements converging docs/TECH_REPORT.md: 68

Honest note on scale: ERA's achievement is a **cross-domain, large-scale** result of "beating humans/ official ensembles on real leaderboards"; this project's achievement is "**on a 10/16 subset of the same benchmark, demonstrating via controlled ablation the marginal contribution of each autonomous capability**," with improvements mostly 0.0x% (significant only in structural events), and the fifth pillar contributing **exactly zero (constructive)**. The two are work of **different magnitude and different claims** — this project's value lies in the **cleanliness and traceability of the methodology**, not in absolute competitive performance.

5. Ablation and Statistical-Rigor Comparison

Aspect	ERA	This project
Core ablation	TS vs Best-of-N (N=1000/128): TS wins for 4 of 5 LLMs and on both problems (only GPT-5 already saturated on batch integration) [ERA p.9,15]	5-stage ladder + sub-stages within each stage; cross-event tier1→tier4 comparison over 15 events docs/ SUMMARY_REPORT.md
Injection ablation	44% (24/55) of recombinations beat both parents; without suggestions it only reaches the ComBat level, literature-guided reaches the BBKNN top of the leaderboard [ERA p.5,29]	fifth-stage single-variable A/B = bit-for-bit identical (constructive no-op) docs/ phase_j_j3_findings.md
Significance testing	none ; main results presented as bar charts and rankings, no confidence intervals reported	paired bootstrap (B=5000) : the full 2→4 ladder over 5 events has all CIs excluding 0; a single tree-search step is significant only on large samples ($n \approx 520k-630k$), while small samples fall within OOF noise docs/ statistical_rigor.md :66-82
Reproducibility gate	sandbox re-runs; manual audit of adherence (nearly all "adhere" to the injected algorithm) [ERA p.34]	bit-wise OOF reproducibility gate : the members of the best solution are retrained from scratch and OOF is compared bit-for-bit before a submission is produced; some reach $\max \Delta OOF =0$ at the bit level docs/ TECH_REPORT.md :45-48

Where this project is actually more rigorous (a highlight to raise with the advisor): ERA's main Kaggle result gives only bar charts, with **no confidence intervals**; this project performs **paired-bootstrap significance** for every stage transition, and **honestly calibrates "the ladder holds in every event" to "trust the cumulative gain, do not over-claim any single 0.0x% step"** (docs/[statistical_rigor.md](#)). This self-questioning of "is a small step merely noise" is an independent methodological contribution of this project. **But with a scope limitation:** the bootstrap currently covers only the 5 cross-season events (with OOF caches); the 10 S3 events lack caches and are not yet included; and the CIs measure only OOF sampling variance, not re-run search (seed/fold/Optuna) variance, so they are a **lower bound**.

6. Honest Gap and Positioning Summary

What this project is "not" (to avoid externally implying equivalence): 1. It is **not** an "LLM writes programs automatically" system — the search is over a fixed GBDT config space, with the LLM not in the mutation loop. 2. The second pillar (external injection) **does not carry load under the current architecture** — ERA's injection is the core mechanism, whereas this project's is a no-op (guided by hand-written seeds). 3. It is **not** validated by a real leaderboard/human percentile — apart from s3e16 it is entirely local CV, with no external yardstick for "reaching expert level." 4. Its scale and domain are far smaller than ERA's (single tabular domain, single driver, hand-written prior seeds).

This project's real value: 1. On a **10/16 subset of the exact same benchmark as ERA**, using **controlled ablation** to segment-by-segment isolate the marginal contribution of "skill-ified workflow / linear self-iteration / tree search" — the **fine-grained attribution the ERA paper did not do**. 2. Its **statistical rigor** (paired bootstrap + bit-level OOF reproducibility gate) is methodologically **more conservative and credible than ERA's main result**. 3. It produced and honestly recorded a **constructive negative finding** about the second pillar (injection no-op), cleanly separating the two claims of "the knowledge did guide" and "the automatic plumbing was consumed" — a demonstration of scientific integrity.

In one sentence: this project is not a replica of ERA nor a scaled-down competitive-performance challenge, but rather **borrowes ERA's two-pillar framework and, on a subset of its same benchmark, performs one clean, traceable, statistically-honest capability-attribution study**; and in the process it honestly delineates the essential gap between the self-built system and ERA.

Appendix: Paths to Truly Align with ERA's Second Pillar Once Wired

To turn "external-idea injection" from a no-op into load-bearing (truly aligning with ERA pillar 2), there are two minimal-change options: 1. **Wire the existing priors:** actually wire `tree['priors']` into `propose_child / SOLO_SEEDS`, so that the injected `[EXT]` ideas change candidate generation (still within the GBDT config space — measurable marginal value, but not equivalent to ERA's code mutation). 2. **Introduce LLM-in-loop code mutation:** within the search loop, let the LLM generate/rewrite feature or modeling code from a prompt (including the injected ideas) — this is the approach isomorphic to ERA, but with substantially larger engineering effort and compute cost, and with package-compatibility risk in an arm64 environment.

The directional question (wire it up vs. accept the honest negative result) is left to be decided; see the closing remarks of `docs/phase_j_j3_findings.md`.